



PUC
CAMPINAS

Monitor: Lucas Soares Gonçalves

Introdução às Tecnologias Web – Engenharia de Software

GUIA DE ESTUDOS DE JAVASCRIPT

GUIA DE JAVASCRIPT

TEORIA

CÓDIGO

RESULTADO

RESPOSTAS COMENTADAS

PARTE I – LINGUAGEM

Tipos, operadores, controle, funções, arrays e objetos.

PARTE II – NAVEGADOR

DOM, eventos, formulários, armazenamento e requisições.

PARTE III – ECOSSISTEMA

Node.js, Express, TypeScript, autenticação e integração de aplicação.

Monitor: Lucas Soares Gonçalves

Professor Orientador: José Guilherme Picolo

Campinas – SP

2026

Sumário

Como usar este guia	1
Mapa do conteúdo	4
I Fundamentos da linguagem	5
1 Valores, variáveis e tipos	6
1.1 Declarações: const, let e var	6
1.2 Tipos primitivos e objetos	7
1.3 Conversão e coerção	7
1.4 Strings e template strings	8
1.5 Números e formatação	8
2 Operadores e expressões	9
2.1 Aritméticos e atribuição	9
2.2 Comparação: prefira igualdade estrita	9
2.3 Lógicos, curto-circuito e nulos	10
2.4 Precedência e parênteses	10
3 Condições e decisões	11
3.1 if, else if e else	11
3.2 Operador ternário	11
3.3 switch para casos discretos	12
4 Repetições e iteração	13
4.1 while e do...while	13
4.2 for com contador	13
4.3 for...of e for...in	14
4.4 break e continue	14
5 Erros, depuração e leitura do fluxo	15
5.1 Três classes de problema	15
5.2 console e debugger	15
5.3 try, catch e finally	15

6	Funções, escopo e composição	17
6.1	Declaração, expressão e arrow function	17
6.2	Parâmetros padrão e retorno antecipado	17
6.3	Escopo e closures	18
6.4	Callbacks e funções de ordem superior	18
7	Arrays e transformação de coleções	19
7.1	Operações essenciais	19
7.2	map, filter, find e reduce	19
8	Objetos, desestruturação e JSON	21
8.1	Desestruturação, spread e rest	21
8.2	Referência e igualdade	22
8.3	JSON não é um objeto JavaScript	22
9	Datas e utilitários da linguagem	23
9.1	Math e aleatoriedade	23
9.2	Date e Intl	23
9.3	Set e Map	24
II	JavaScript no navegador	25
10	DOM: a página como árvore de objetos	26
10.1	Selecionar elementos	26
10.2	Ler e alterar conteúdo	27
10.3	Criar, inserir e remover	27
10.4	Renderizar listagens e tabelas com segurança	27
11	Eventos e interação	29
11.1	addEventListener e o objeto event	29
11.2	Propagação e delegação	29
11.3	Formulários e preventDefault	30
11.4	FormData, normalização e envio	30
11.5	Validação: interface e regra de negócio	32
12	Estado e armazenamento no navegador	33
12.1	localStorage	33
12.2	Um padrão de estado e renderização	34

12.3 Busca, filtros e ordenação	34
12.4 Estados de carregamento, vazio, erro e sucesso	35
13 Tempo, callbacks e promessas	36
13.1 Temporizadores e ordem de execução	36
13.2 Promise e seus estados	36
13.3 async e await	37
14 HTTP, fetch e JSON	38
14.1 Ler dados com GET	38
14.2 Busca, filtros, ordenação e paginação na URL	38
14.3 Enviar JSON com POST	39
14.4 Atualização parcial e cancelamento lógico	39
14.5 Métodos e status essenciais	40
14.6 Um cliente HTTP reutilizável	40
14.7 Abortar requisições e evitar resultados antigos	40
14.8 Consumir um serviço externo	41
14.9 Concorrência com Promise.all	41
15 Módulos e organização	43
15.1 export e import	43
15.2 Um desenho por camadas	43
III Ecossistema e aplicação	45
16 Node.js, npm e APIs com Express	46
16.1 package.json, node_modules e lockfile	46
16.2 Scripts npm e módulos	46
16.3 Servidor HTTP com Express	47
16.4 Validar entrada de uma rota	47
16.5 Rotas, middlewares e camadas	48
16.6 Persistência, histórico e transações	49
16.7 Parâmetros de rota e de consulta	49
16.8 Operação segura e integração	50

17 TypeScript sem mistério	54
17.1 Tipos básicos e inferência	54
17.2 Objetos, aliases e união	54
17.3 Narrowing e unknown	55
17.4 Tipos internos e validação de entrada	55
17.5 Configuração estrita	56
17.6 Módulos e compilação	57
18 Bootstrap e JavaScript de componentes	58
18.1 Arquivos minificados	58
18.2 Data attributes e API JavaScript	58
19 Exemplo integrado: painel de registros	59
19.1 Estrutura	59
19.2 Modelo e regras	59
19.3 Interface HTML	60
19.4 Cliente HTTP e renderização	61
19.5 Carregar, cadastrar e cancelar	62
19.6 API Express em TypeScript	63
19.7 Executar e observar	64
19.8 Leitura guiada de ponta a ponta	64
20 Qualidade, segurança e diagnóstico	66
20.1 Erros frequentes	66
20.2 Segurança básica	67
20.3 Acessibilidade em interfaces dinâmicas	67
20.4 Checklist antes de considerar pronto	67
21 Revisão com respostas comentadas	69
22 Referência rápida de JavaScript	72
Fontes e créditos	74

Como usar este guia

JavaScript é a linguagem que transforma uma página estática em um programa: lê dados, toma decisões, altera a interface, conversa com serviços e também pode executar fora do navegador. Este material foi organizado para que você não memorize linhas soltas. A meta é compreender o que ocorre entre a entrada e a saída.

O MAPA MENTAL PRINCIPAL

Entrada fornece dados; o **programa** transforma esses dados; a **saída** torna o resultado observável. No navegador, a entrada pode vir de um campo ou clique. No servidor, pode vir de uma requisição HTTP.

A sequência de estudo

Em quase todos os exemplos, procure responder quatro perguntas:

1. Quais valores entram no programa e de que tipo eles são?
2. Qual expressão, decisão, repetição ou função os transforma?
3. Onde o resultado fica armazenado?
4. Onde o resultado aparece: console, página ou resposta HTTP?

JAVASCRIPT

```
1 const nome = 'Lia'           // entrada
2 const mensagem = `Olá, ${nome}` // transformação
3 console.log(mensagem)       // saída
```

SAÍDA NO CONSOLE

Olá, Lia

O comentário começa com `//` e não é executado. `const` cria uma ligação que não será reatribuída; a template string, entre crases, insere o valor de `nome`; `console.log()` registra o resultado.

Onde executar JavaScript

Console do navegador

Abra as ferramentas do desenvolvedor – normalmente com `F12` ou `Option + Command + I` no macOS – e escolha a aba *Console*. É o ambiente mais rápido para testar uma expressão.

JAVASCRIPT

```
1 2 + 3 * 4
```

SAÍDA

14

Dentro de uma página

Para um arquivo externo, prefira `defer`. O navegador baixa o script sem bloquear a leitura do HTML e o executa depois que o documento foi analisado.

HTML

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Meu primeiro script</title>
7   <script src="app.js" defer></script>
8 </head>
9 <body>
10  <h1 id="titulo">JavaScript</h1>
11 </body>
12 </html>
```

JAVASCRIPT

```
1 const titulo = document.querySelector('#titulo')
2 titulo.textContent = 'JavaScript funcionando!'
```

RESULTADO NO NAVEGADOR

O título visível muda de "JavaScript" para **JavaScript funcionando!**.

No Node.js

Node.js executa JavaScript fora do navegador. Um arquivo chamado `app.js` pode ser iniciado no terminal:

TERMINAL

```
node app.js
```

O Node não oferece `document` nem `window`; esses objetos pertencem ao navegador. Em compensação, oferece APIs de arquivos, processos, rede e módulos.

Como os blocos do guia funcionam

- **Código:** trecho executável ou estrutura mínima.
- **Saída:** o que aparece no console, na página ou em uma resposta.

- **Conceito:** mecanismo que explica por que o código funciona.
- **Atenção:** erro frequente ou diferença importante.
- **Pergunta:** verificação curta; a resposta vem no mesmo quadro.

LINGUAGEM NÃO É AMBIENTE

JavaScript é a linguagem. DOM, `fetch`, temporizadores e armazenamento são APIs do navegador. Node.js é outro ambiente de execução. TypeScript adiciona tipos e é convertido para JavaScript. Bootstrap é um framework de interface que usa CSS e, em alguns componentes, JavaScript.

PERGUNTA: Preciso decorar todos os métodos?

RESPOSTA Não. Memorize os modelos mentais – tipo, fluxo, função, coleção, evento e requisição – e aprenda a consultar a documentação. Saber prever o resultado de um pequeno trecho é mais valioso do que decorar uma lista extensa.

Mapa do conteúdo

Parte	Competência construída
Fundamentos	Representar valores, escrever expressões, controlar o fluxo e decompor problemas em funções.
Dados	Trabalhar com arrays, objetos, strings, datas e transformações de coleção.
Navegador	Selecionar o DOM, tratar formulários, renderizar listagens, filtrar dados e comunicar estados da interface.
Assíncrono	Consumir JSON, compreender promessas, montar consultas, chamar serviços externos e tratar falhas.
Ecossistema	Entender Node.js, npm, API HTTP, Express, TypeScript, autenticação, permissões e Bootstrap.
Integração	Conectar uma interface administrativa a uma API organizada em camadas.

PARTE I

Fundamentos da linguagem

1 Valores, variáveis e tipos

Um programa manipula valores. Cada valor tem um tipo, e o tipo determina quais operações fazem sentido. Somar números é diferente de concatenar textos; comparar um número com uma string pode produzir coerção inesperada.

1.1 Declarações: `const`, `let` e `var`

Use `const` como padrão. Use `let` quando a variável realmente precisar receber outro valor. `var` existe por compatibilidade e tem regras de escopo e elevação mais propensas a confusão.

JAVASCRIPT

```
1 const curso = 'JavaScript'
2 let aulasConcluidas = 3
3
4 aulasConcluidas = aulasConcluidas + 1
5 console.log(curso, aulasConcluidas)
```

SAÍDA

JavaScript 4

CONST NÃO TORNA UM OBJETO IMUTÁVEL

`const` impede que a variável aponte para outro valor. As propriedades internas de um objeto ou os itens de um array ainda podem mudar.

JAVASCRIPT

```
1 const aluno = { nome: 'Bia', pontos: 8 }
2 aluno.pontos = 9           // permitido
3 // aluno = {}             // TypeError: reatribuição não permitida
```

1.2 Tipos primitivos e objetos

Tipo	Uso	Exemplo
string	Texto e caracteres.	'PUC Campinas'
number	Inteiros, decimais, NaN e infinitos.	42; 3.14
bigint	Inteiros maiores que o limite seguro de number.	9007199254740993n
boolean	Estado lógico verdadeiro ou falso.	true
undefined	Ausência ainda não definida.	let resposta
null	Ausência definida intencionalmente.	null
symbol	Identificador único, comum em APIs avançadas.	Symbol('id')
object	Coleções e estruturas, incluindo arrays.	ativo: true

JAVASCRIPT

```

1 console.log(typeof 'web') // string
2 console.log(typeof 10) // number
3 console.log(typeof true) // boolean
4 console.log(typeof undefined) // undefined
5 console.log(typeof null) // object: peculiaridade histórica
6 console.log(Array.isArray([])) // true

```

NaN SIGNIFICA “NÃO É UM NÚMERO VÁLIDO”

NaN pertence ao tipo number. Use `Number.isNaN(valor)` para verificar se uma conversão numérica falhou.

1.3 Conversão e coerção

Conversão explícita deixa a intenção visível. Coerção ocorre automaticamente, às vezes de forma surpreendente.

JAVASCRIPT

```

1 const texto = '12.5'
2 const numero = Number(texto)
3
4 console.log(numero + 2)
5 console.log(String(2026))
6 console.log(Boolean(''))

```

SAÍDA

```

14.5
"2026"
false

```

Valores *falsy*: `false`, `0`, `-0`, `0n`, string vazia, `null`, `undefined` e `NaN`. Os demais são *truthy*; inclusive `'0'` e `[]`.

ENTRADAS DE FORMULÁRIO SÃO STRINGS

Mesmo quando o campo usa `type='number'`, sua propriedade `value` é texto. Converta com `Number(campo.value)` ou use `campo.valueAsNumber` quando apropriado.

1.4 Strings e template strings

Strings podem usar aspas simples, duplas ou crases. Crases permitem interpolação e múltiplas linhas.

JAVASCRIPT

```
1 const nome = 'Ravi'
2 const nota = 8.75
3 const resumo = `${nome} obteve ${nota.toFixed(1)}.`
4
5 console.log(resumo)
6 console.log(nome.length)
7 console.log(nome.toUpperCase())
```

SAÍDA

```
Ravi obteve 8.8.
4
RAVI
```

Métodos úteis incluem `includes()`, `startsWith()`, `slice()`, `trim()`, `replace()` e `split()`.

1.5 Números e formatação

JAVASCRIPT

```
1 const preco = 1299.9
2
3 console.log(preco.toFixed(2))
4 console.log(preco.toLocaleString('pt-BR', {
5   style: 'currency',
6   currency: 'BRL'
7 })))
```

SAÍDA

```
1299.90
R$ 1.299,90
```

PONTO FLUTUANTE

Números decimais são representados em binário. Por isso, `0.1 + 0.2` não é exatamente `0.3`. Para dinheiro, armazene centavos inteiros ou use uma biblioteca decimal apropriada.

2 Operadores e expressões

Uma expressão produz um valor. Operadores combinam valores e definem como esse resultado será calculado.

2.1 Aritméticos e atribuição

Operador	Significado
+ - * /	Soma, subtração, multiplicação e divisão.
%	Resto da divisão; útil para paridade e ciclos.
**	Potenciação.
++ --	Incremento ou decremento de uma unidade.
+= -= *= /=	Opera e reatribui.

JAVASCRIPT

```
1 let pontos = 10
2 pontos += 5
3
4 console.log(7 % 2)
5 console.log(2 ** 5)
6 console.log(pontos)
```

SAÍDA

```
1
32
15
```

2.2 Comparação: prefira igualdade estrita

=== e !== comparam valor e tipo. == e != permitem coerção automática.

JAVASCRIPT

```
1 console.log(5 === 5)    // true
2 console.log(5 === '5')  // false
3 console.log(5 == '5')   // true: houve coerção
```

Como regra de projeto, prefira os operadores estritos. Isso torna o resultado previsível.

2.3 Lógicos, curto-circuito e nulos

JAVASCRIPT

```
1 const idade = 20
2 const temIngresso = true
3 const podeEntrar = idade >= 18 && temIngresso
4
5 const apelido = ''
6 const exibidoComOu = apelido || 'Visitante'
7 const exibidoComNulo = apelido ?? 'Visitante'
8
9 console.log(podeEntrar, exibidoComOu, exibidoComNulo)
```

SAÍDA

```
true Visitante ""
```

`&&` exige que ambos sejam truthy; `||` devolve o primeiro truthy; `??` usa o valor da direita somente quando o da esquerda é `null` ou `undefined`. Por isso a string vazia é preservada por `??`.

2.4 Precedência e parênteses

Multiplicação acontece antes da soma; comparação, depois da aritmética; negação antes de `&&`, que vem antes de `||`. Use parênteses quando eles melhorarem a leitura.

JAVASCRIPT

```
1 const media = (7 + 8 + 9) / 3
2 const aprovado = media >= 6 && media <= 10
3 console.log(media, aprovado)
```

SAÍDA

```
8 true
```

PERGUNTA: Qual é a diferença entre `Number('10')` e `parseInt('10px')`?

RESPOSTA `Number('10')` converte toda a string; `parseInt('10px')` lê o prefixo inteiro e para ao encontrar texto. `Number('10px')` produz `NaN`. Para entradas que devem ser inteiramente numéricas, `Number()` costuma comunicar melhor a validação esperada.

3 Condições e decisões

Uma condição escolhe qual caminho executar. O teste deve resultar em um valor truthy ou falsy, mas escrever comparações explícitas torna regras de negócio mais legíveis.

3.1 if, else if e else

JAVASCRIPT

```
1 const media = 7.2
2
3 if (media >= 7) {
4   console.log('Aprovado')
5 } else if (media >= 5) {
6   console.log('Recuperação')
7 } else {
8   console.log('Reprovado')
9 }
```

SAÍDA

Aprovado

Somente o primeiro ramo verdadeiro é executado. As chaves delimitam o bloco e evitam que uma alteração futura mude o fluxo por acidente.

3.2 Operador ternário

Use o ternário para escolher entre dois valores curtos, não para esconder uma sequência complexa.

JAVASCRIPT

```
1 const saldo = 120
2 const status = saldo >= 0 ? 'regular' : 'devedor'
3 console.log(status)
```

SAÍDA

regular

3.3 switch para casos discretos

JAVASCRIPT

```
1 const dia = 2
2 let nome
3
4 switch (dia) {
5   case 1:
6     nome = 'domingo'
7     break
8   case 2:
9     nome = 'segunda-feira'
10    break
11  default:
12    nome = 'outro dia'
13 }
14
15 console.log(nome)
```

SAÍDA

segunda-feira

Sem `break`, a execução continua no próximo caso – comportamento chamado *fall-through*. Isso pode ser intencional, mas deve estar claro.

PERGUNTA: Quando switch é melhor do que if/else?

RESPOSTA Quando uma mesma expressão é comparada a vários valores discretos. Para faixas como “idade entre 18 e 59”, `if/else` é mais direto.

4 Repetições e iteração

Um laço repete um bloco enquanto uma condição for verdadeira ou para cada item de uma coleção. Todo laço precisa de uma forma de terminar.

4.1 while e do...while

JAVASCRIPT

```
1 let contador = 1
2
3 while (contador <= 3) {
4   console.log(`Passo ${contador}`)
5   contador++
6 }
```

SAÍDA

```
Passo 1
Passo 2
Passo 3
```

`while` testa antes de executar. `do...while` executa uma vez e testa depois.

JAVASCRIPT

```
1 let tentativa = 1
2
3 do {
4   console.log(`Tentativa ${tentativa}`)
5   tentativa++
6 } while (tentativa <= 2)
```

4.2 for com contador

O `for` reúne inicialização, teste e atualização.

JAVASCRIPT

```
1 let soma = 0
2
3 for (let i = 1; i <= 10; i++) {
4   const sinal = i % 2 === 0 ? -1 : 1
5   soma += sinal * (i / (i ** 2))
6 }
7
8 console.log(soma.toFixed(6))
```

SAÍDA

0.645635

O sinal alterna: positivo quando `i` é ímpar e negativo quando é par. A fração `i / i ** 2` equivale a `1 / i`; mantê-la na forma original ajuda a reconhecer a fórmula.

4.3 for...of e for...in

JAVASCRIPT

```
1 const notas = [8, 6.5, 9]
2
3 for (const nota of notas) {
4   console.log(nota)
5 }
```

`for...of` percorre valores de iteráveis. `for...in` percorre nomes de propriedades enumeráveis e é mais adequado a objetos simples; em arrays, pode incluir chaves inesperadas e devolve índices como strings.

4.4 break e continue

JAVASCRIPT

```
1 for (let n = 1; n <= 8; n++) {
2   if (n === 3) continue
3   if (n === 7) break
4   console.log(n)
5 }
```

SAÍDA

1 2 4 5 6

`continue` pula para a próxima iteração; `break` encerra o laço.

LAÇO INFINITO

Se a condição nunca se torna falsa, o programa fica preso. Verifique a variável de controle, os limites e se a atualização realmente ocorre em todos os caminhos.

5 Erros, depuração e leitura do fluxo

Erros são informações sobre uma suposição quebrada. A mensagem indica o tipo; a pilha mostra o caminho de chamadas; a linha aponta onde o problema foi observado – nem sempre onde nasceu.

5.1 Três classes de problema

Categoria	Exemplo
Sintaxe	Chave, parêntese ou aspas não fechadas; o código não é analisado.
Execução	Acessar uma variável inexistente ou chamar algo que não é função.
Lógica	O programa executa, mas calcula ou mostra o resultado errado.

5.2 console e debugger

JAVASCRIPT

```
1 function calcularMedia(notas) {  
2   console.table(notas)  
3   const total = notas.reduce((soma, nota) => soma + nota, 0)  
4   debugger  
5   return total / notas.length  
6 }
```

Com as ferramentas abertas, `debugger` pausa a execução. Inspecione variáveis, avance linha a linha e confirme qual ramo foi percorrido. Remova pausas antes de publicar.

5.3 try, catch e finally

JAVASCRIPT

```
1 function lerConfiguracao(texto) {  
2   try {  
3     return JSON.parse(texto)  
4   } catch (erro) {  
5     console.error('JSON inválido:', erro.message)  
6     return null  
7   } finally {  
8     console.log('Leitura encerrada')  
9   }  
10 }
```

Capture apenas erros que você consegue tratar. Não use `catch` vazio: ele esconde o diagnóstico.

PERGUNTA: Por que `console.log` não substitui um depurador?

RESPOSTA Porque o depurador preserva o estado exato e permite avançar pela execução. Logs continuam úteis para observar dados, mas exigem antecipar o que imprimir e podem alterar o tempo de código assíncrono.

6 Funções, escopo e composição

Uma função dá nome a uma transformação. Ela recebe parâmetros, executa um corpo e pode devolver um valor com `return`. Bons nomes reduzem a necessidade de comentários.

6.1 Declaração, expressão e arrow function

JAVASCRIPT

```
1 function somar(a, b) {  
2   return a + b  
3 }  
4  
5 const dobro = function (numero) {  
6   return numero * 2  
7 }  
8  
9 const quadrado = numero => numero ** 2  
10  
11 console.log(somar(2, 3), dobro(4), quadrado(5))
```

SAÍDA

5 8 25

Declarações de função são lidas integralmente. Expressões são atribuídas quando a execução chega à linha. Arrow functions não criam seu próprio `this`; isso é útil em callbacks, mas não as torna substitutas universais de métodos tradicionais.

6.2 Parâmetros padrão e retorno antecipado

JAVASCRIPT

```
1 function dividir(dividendo, divisor = 1) {  
2   if (divisor === 0) return null  
3   return dividendo / divisor  
4 }  
5  
6 console.log(dividir(10, 2))  
7 console.log(dividir(10, 0))
```

SAÍDA

5
null

O retorno antecipado trata a exceção de domínio e evita aninhamento excessivo.

6.3 Escopo e closures

`let` e `const` respeitam o bloco. Uma closure é uma função que conserva acesso ao ambiente em que foi criada.

JAVASCRIPT

```
1 function criarContador() {  
2   let valor = 0  
3  
4   return () => {  
5     valor++  
6     return valor  
7   }  
8 }  
9  
10 const proximo = criarContador()  
11 console.log(proximo(), proximo(), proximo())
```

SAÍDA

1 2 3

Cada chamada a `criarContador()` produz um novo estado privado.

6.4 Callbacks e funções de ordem superior

Uma função é um valor: pode ser armazenada, recebida e devolvida.

JAVASCRIPT

```
1 function executarOperacao(a, b, operacao) {  
2   return operacao(a, b)  
3 }  
4  
5 const resultado = executarOperacao(6, 4, (x, y) => x - y)  
6 console.log(resultado)
```

SAÍDA

2

7 Arrays e transformação de coleções

Array é uma coleção ordenada, indexada a partir de zero. `length` informa a quantidade de itens.

7.1 Operações essenciais

JAVASCRIPT

```
1 const tecnologias = ['HTML', 'CSS']
2
3 tecnologias.push('JavaScript')
4 tecnologias.unshift('Web')
5 const removida = tecnologias.pop()
6
7 console.log(tecnologias)
8 console.log(removida)
```

SAÍDA

```
['Web', 'HTML', 'CSS']
JavaScript
```

`push/pop` atuam no fim; `unshift/shift`, no início. `slice` copia uma faixa sem alterar o original; `splice` remove ou insere alterando o array.

7.2 map, filter, find e reduce

JAVASCRIPT

```
1 const notas = [4, 7, 8, 5, 9]
2
3 const ajustadas = notas.map(nota => Math.min(nota + 1, 10))
4 const aprovadas = ajustadas.filter(nota => nota >= 6)
5 const primeiraAlta = ajustadas.find(nota => nota >= 9)
6 const soma = ajustadas.reduce((total, nota) => total + nota, 0)
7
8 console.log(ajustadas)
9 console.log(aprovadas)
10 console.log(primeiraAlta)
11 console.log(soma)
```

SAÍDA

```
[5, 8, 9, 6, 10]
[8, 9, 6, 10]
9
38
```

Método	Pergunta que responde
map	Como fica cada item transformado?
filter	Quais itens satisfazem a condição?
find	Qual é o primeiro item que satisfaz a condição?
some/every	Algum item/todos os itens satisfazem?
reduce	Como combinar todos os itens em um único acumulador?

SORT ORDENA COMO TEXTO POR PADRÃO

`[10, 2, 5].sort()` produz `[10, 2, 5]`. Para números, use `.sort((a, b) => a - b)`. O método também altera o array original; copie antes com `[...valores]` se precisar preservá-lo.

8 Objetos, desestruturação e JSON

Objetos agrupam propriedades nomeadas. Use ponto quando a chave é conhecida; colchetes quando ela está em uma variável ou contém caracteres especiais.

JAVASCRIPT

```
1 const curso = {
2   nome: 'JavaScript',
3   cargaHoraria: 40,
4   ativo: true,
5   apresentar() {
6     return `${this.nome}: ${this.cargaHoraria}h`
7   }
8 }
9
10 console.log(curso.nome)
11 console.log(curso['ativo'])
12 console.log(curso.apresentar())
```

SAÍDA

```
JavaScript
true
JavaScript: 40h
```

8.1 Desestruturação, spread e rest

JAVASCRIPT

```
1 const aluno = { nome: 'Caio', nota: 9, turma: 'A' }
2 const { nome, nota } = aluno
3 const atualizado = { ...aluno, nota: 9.5 }
4
5 function media(...valores) {
6   const total = valores.reduce((soma, valor) => soma + valor, 0)
7   return total / valores.length
8 }
9
10 console.log(nome, nota)
11 console.log(atualizado)
12 console.log(media(7, 8, 9))
```

Spread expande itens/propriedades; rest reúne argumentos restantes. A cópia com spread é rasa: objetos internos continuam compartilhados.

8.2 Referência e igualdade

JAVASCRIPT

```
1 const a = { valor: 1 }  
2 const b = a  
3 const c = { valor: 1 }  
4  
5 console.log(a === b)  
6 console.log(a === c)
```

SAÍDA

```
true  
false
```

Objetos são comparados por identidade, não pelo conteúdo. `b` aponta para o mesmo objeto de `a`; `c` é outro objeto.

8.3 JSON não é um objeto JavaScript

JSON é um formato textual de intercâmbio. Não aceita comentários, funções, `undefined` nem chaves sem aspas.

JSON

```
1 {  
2   "nome": "Lia",  
3   "nota": 9,  
4   "aprovada": true  
5 }
```

JAVASCRIPT

```
1 const texto = '{"nome":"Lia","nota":9}'  
2 const dados = JSON.parse(texto)  
3 const novamenteTexto = JSON.stringify(dados)  
4  
5 console.log(dados.nome)  
6 console.log(novamenteTexto)
```

SAÍDA

```
Lia  
{"nome":"Lia","nota":9}
```

Uma chamada executa um único `return`, mas esse retorno pode ser uma estrutura com vários dados. Para devolver vários resultados, prefira um objeto como `{ soma, media, maior }` quando eles têm nomes, ou um array quando a ordem é suficiente.

9 Datas e utilitários da linguagem

9.1 Math e aleatoriedade

JAVASCRIPT

```
1 console.log(Math.round(4.6)) // 5
2 console.log(Math.floor(4.9)) // 4
3 console.log(Math.ceil(4.1)) // 5
4 console.log(Math.max(3, 8, 5))
5
6 const inteiroDe1A6 = Math.floor(Math.random() * 6) + 1
```

`Math.random()` produz valor entre 0 inclusive e 1 exclusivo; não é apropriado para criptografia. Para identificadores, use `crypto.randomUUID()` quando disponível.

9.2 Date e Intl

JAVASCRIPT

```
1 const agora = new Date()
2 const formato = new Intl.DateTimeFormat('pt-BR', {
3   dateStyle: 'long',
4   timeStyle: 'short'
5 })
6
7 console.log(formato.format(agora))
```

Datas representam um instante em milissegundos desde 1970. Exibição depende de fuso e localidade. Strings no formato ISO, como `2026-08-29T12:00:00Z`, tornam o fuso explícito com Z.

Uma data de calendário, como um prazo sem horário, não é necessariamente um instante UTC. Preserve o texto validado no formato `AAAA-MM-DD` ou construa uma data local com componentes; usar diretamente `new Date('2026-09-05')` pode produzir outro dia ao formatar em alguns fusos.

JAVASCRIPT

```
1 function dataLocal(dataISO) {
2   const [ano, mes, dia] = dataISO.split('-').map(Number)
3   return new Date(ano, mes - 1, dia)
4 }
5
6 const prazo = dataLocal('2026-09-05')
7 console.log(new Intl.DateTimeFormat('pt-BR').format(prazo))
```

No contrato da API, diferencie claramente data sem horário, data/hora com fuso e instante UTC. O servidor deve gerar datas de criação e atualização; não confie no relógio do navegador para auditoria.

9.3 Set e Map

JAVASCRIPT

```
1 const tags = new Set(['js', 'dom', 'js'])
2 tags.add('node')
3
4 const notas = new Map()
5 notas.set('Lia', 9)
6 notas.set('Ravi', 8)
7
8 console.log([...tags])
9 console.log(notas.get('Lia'))
```

SAÍDA

```
['js', 'dom', 'node']
9
```

Set mantém valores únicos. Map associa chaves de qualquer tipo a valores e possui API própria; um objeto simples continua adequado para registros com campos conhecidos.

O QUE FICA PARA CONSULTA POSTERIOR

Classes, protótipos e expressões regulares são recursos úteis, mas não são pré-requisitos para organizar uma aplicação web introdutória. Objetos, funções e módulos cobrem a maior parte da estrutura necessária. Consulte esses temas quando surgir um problema que realmente os justifique.

PARTE II

JavaScript no navegador

10 DOM: a página como árvore de objetos

O DOM (Document Object Model) é a representação em memória que o navegador constrói a partir do HTML. JavaScript não altera diretamente o arquivo `.html`; altera os objetos dessa árvore, e o navegador redesenha a interface.

UMA ROTA NA ÁRVORE

`window` → `document` → `html` → `body` → `main` → `p`. `window` representa a janela; `document`, o documento carregado; cada elemento HTML torna-se um nó.

10.1 Selecionar elementos

HTML

```
1 <h1 id="titulo">Cursos</h1>
2 <ul class="lista">
3   <li>HTML</li>
4   <li>CSS</li>
5   <li>JavaScript</li>
6 </ul>
```

JAVASCRIPT

```
1 const titulo = document.querySelector('#titulo')
2 const lista = document.querySelector('.lista')
3 const itens = document.querySelectorAll('.lista li')
4
5 console.log(titulo.textContent)
6 console.log(itens.length)
```

SAÍDA

```
Cursos
3
```

`querySelector()` devolve o primeiro elemento compatível ou `null`. `querySelectorAll()` devolve uma `NodeList`. Também existem `getElementById()`, `getElementsByName()` e `getElementsByClassName()`, mas seletores CSS oferecem uma linguagem consistente.

INDICA ID; . INDICA CLASSE

Em `querySelector('#mensagem')`, `#` seleciona o atributo `id`. Em `querySelector('.mensagem')`, o ponto seleciona `class`. Sem o símbolo, procura-se uma tag com aquele nome.

10.2 Ler e alterar conteúdo

JAVASCRIPT

```
1 titulo.textContent = 'Trilha Front-end'
2 lista.classList.add('lista--destacada')
3 lista.dataset.total = String(itens.length)
```

`textContent` trata o valor como texto. `innerHTML` interpreta marcação e deve ser reservado a conteúdo controlado.

EVITE INJETAR ENTRADA COM INNERHTML

Se um texto veio do usuário ou de fonte não confiável, use `textContent`. Inserir esse valor em `innerHTML` pode permitir execução de marcação e scripts maliciosos.

10.3 Criar, inserir e remover

JAVASCRIPT

```
1 const item = document.createElement('li')
2 item.textContent = 'Node.js'
3 lista.append(item)
4
5 const primeiro = lista.firstChild
6 primeiro.remove()
```

RESULTADO NO NAVEGADOR

A lista passa a conter CSS, JavaScript e Node.js. O item HTML foi removido e o novo item foi acrescentado ao final.

10.4 Renderizar listagens e tabelas com segurança

Uma listagem recebida de uma API precisa ser transformada em nós do DOM. Crie a estrutura com os elementos corretos e insira os valores com `textContent`; assim a tabela continua semântica e textos externos não são interpretados como HTML.

HTML

```
1 <table>
2   <caption>Registros encontrados</caption>
3   <thead>
4     <tr><th scope="col">Título</th><th scope="col">Situação</th></tr>
5   </thead>
6   <tbody id="registros"></tbody>
7 </table>
```

JAVASCRIPT

```
1  const corpo = document.querySelector('#registros')
2
3  function criarCelula(texto) {
4    const celula = document.createElement('td')
5    celula.textContent = texto
6    return celula
7  }
8
9  function renderizarRegistros(registros) {
10   const fragmento = document.createDocumentFragment()
11
12   for (const registro of registros) {
13     const linha = document.createElement('tr')
14     linha.dataset.id = registro.id
15     linha.append(
16       criarCelula(registro.titulo),
17       criarCelula(registro.situacao)
18     )
19     fragmento.append(linha)
20   }
21
22   corpo.replaceChildren(fragmento)
23 }
```

`DocumentFragment` reúne os nós antes da inserção. `replaceChildren()` evita duplicar linhas em uma nova renderização. A propriedade `dataset.id` mantém o identificador disponível para ações e delegação de eventos.

11 Eventos e interação

Evento é uma ocorrência observável: clique, digitação, envio de formulário, carregamento, foco ou mudança. Um listener registra uma função para responder quando a ocorrência acontecer.

11.1 addEventListener e o objeto event

HTML

```
1 <button id="saudar" type="button">Saudar</button>
2 <p id="resposta" aria-live="polite"></p>
```

JAVASCRIPT

```
1 const botao = document.querySelector('#saudar')
2 const resposta = document.querySelector('#resposta')
3
4 botao.addEventListener('click', event => {
5   resposta.textContent = `Clique recebido de ${event.currentTarget.id}.`
6 })
```

RESULTADO APÓS O CLIQUE

Clique recebido de saudar.

`event.target` é o nó onde o evento nasceu; `event.currentTarget` é o nó cujo listener está executando. A diferença importa quando há elementos aninhados.

11.2 Propagação e delegação

Eventos normalmente sobem pela árvore. Delegação usa um único listener no ancestral para tratar muitos filhos, inclusive criados depois.

JAVASCRIPT

```
1 lista.addEventListener('click', event => {
2   const item = event.target.closest('li')
3   if (!item || !lista.contains(item)) return
4   item.classList.toggle('selecionado')
5 })
```

11.3 Formulários e preventDefault

HTML

```
1 <form id="form-nota">
2   <label for="nota">Nota</label>
3   <input id="nota" name="nota" type="number" min="0" max="10" required>
4   <button type="submit">Classificar</button>
5 </form>
6 <p id="resultado" aria-live="polite"></p>
```

JAVASCRIPT

```
1 const formulario = document.querySelector('#form-nota')
2 const campoNota = document.querySelector('#nota')
3 const resultado = document.querySelector('#resultado')
4
5 formulario.addEventListener('submit', event => {
6   event.preventDefault()
7
8   const nota = campoNota.valueAsNumber
9   const status = nota >= 7 ? 'Aprovado' : 'Em recuperação'
10  resultado.textContent = status
11 })
```

RESULTADO PARA A NOTA 8

Aprovado

`preventDefault()` impede o envio tradicional e o recarregamento. O listener deve ficar no evento `submit`, não apenas no clique do botão, para funcionar também com Enter.

11.4 FormData, normalização e envio

`FormData` coleta controles que possuem `name`. O objeto ainda deve ser normalizado: campos chegam como strings, espaços podem precisar ser removidos e caixas não marcadas não aparecem no conjunto.

JAVASCRIPT

```
1 formulario.addEventListener('submit', async event => {
2   event.preventDefault()
3
4   if (!formulario.checkValidity()) {
5     formulario.reportValidity()
6     return
7   }
8
9   const formularioDados = new FormData(formulario)
10  const dados = Object.fromEntries(formularioDados)
11
12  const registro = {
13    titulo: dados.titulo.trim(),
14    prioridade: dados.prioridade,
```

```
15     prazo: dados.prazo || null,  
16     receberAvisos: formularioDados.has('receberAvisos')  
17   }  
18  
19   await salvarRegistro(registro)  
20 }
```

O uso de `checkValidity()` aproveita as restrições do HTML; regras dependentes do domínio continuam necessárias. Desabilite o botão durante o envio para evitar duplicidade, mas reabilite-o em `finally`.

11.5 Validação: interface e regra de negócio

HTML pode validar presença, tipo, faixa e padrão. JavaScript completa regras que dependem de vários campos.

SEPARAÇÃO DE RESPONSABILIDADES

A função que lê a interface não precisa decidir a regra do domínio. Uma função pura pode receber números e devolver um resultado; outra camada coleta campos e mostra a mensagem. Isso facilita testes e reaproveitamento no servidor.

JAVASCRIPT

```
1 function lerNumero(campo) {  
2   const valor = campo.valueAsNumber  
3   if (!Number.isFinite(valor)) {  
4     campo.setCustomValidity('Informe um número válido.')  
5     return null  
6   }  
7   campo.setCustomValidity('')  
8   return valor  
9 }
```

Ao identificar um erro, mantenha texto visível associado ao campo. JavaScript pode atualizar `aria-invalid`, remover `hidden` da mensagem e devolver foco ao primeiro controle inválido. Não comunique erro somente por cor.

12 Estado e armazenamento no navegador

Estado é a informação que muda enquanto o programa executa. Quando a página recarrega, variáveis desaparecem; para dados pequenos e não sensíveis, o navegador oferece `localStorage` e `sessionStorage`.

12.1 `localStorage`

JAVASCRIPT

```
1 const preferencias = { tema: 'escuro', fonte: 18 }
2
3 localStorage.setItem('preferencias', JSON.stringify(preferencias))
4
5 const texto = localStorage.getItem('preferencias')
6 const salvas = texto ? JSON.parse(texto) : null
7
8 console.log(salvas?.tema)
```

SAÍDA

escuro

`localStorage` persiste entre sessões do mesmo site; `sessionStorage`, somente durante a aba. Ambos guardam strings e são síncronos.

NÃO GUARDE SEGREDOS

Código executado na mesma origem pode ler esse armazenamento. Não salve senhas, tokens de longa duração ou dados sensíveis. Para volumes maiores e consultas estruturadas, considere IndexedDB.

12.2 Um padrão de estado e renderização

JAVASCRIPT

```
1  const estado = {
2    tarefas: []
3  }
4
5  function adicionarTarefa(texto) {
6    estado.tarefas.push({ id: crypto.randomUUID(), texto, concluida: false })
7    renderizar()
8  }
9
10 function renderizar() {
11   lista.replaceChildren()
12   for (const tarefa of estado.tarefas) {
13     const item = document.createElement('li')
14     item.textContent = tarefa.texto
15     lista.append(item)
16   }
17 }
```

O estado é a fonte da verdade; `renderizar()` projeta o estado no DOM. Evite manter duas versões independentes da mesma informação.

12.3 Busca, filtros e ordenação

Filtros locais são adequados quando o conjunto já está carregado e é pequeno. Para muitos registros, envie os critérios à API para filtrar, ordenar e paginar no servidor.

JAVASCRIPT

```
1  function consultarLocalmente(registros, { texto = '', situacao = '', ordem }) {
2    const termo = texto.trim().toLocaleLowerCase('pt-BR')
3
4    const filtrados = registros.filter(registro => {
5      const combinaTexto = !termo ||
6        registro.titulo.toLocaleLowerCase('pt-BR').includes(termo)
7      const combinaSituacao = !situacao || registro.situacao === situacao
8      return combinaTexto && combinaSituacao
9    })
10
11    return filtrados.toSorted((a, b) => {
12      if (ordem === 'titulo') return a.titulo.localeCompare(b.titulo, 'pt-BR')
13      if (ordem === 'prazo') return (a.prazo ?? '').localeCompare(b.prazo ?? '')
14      return 0
15    })
16 }
```

`filter()` seleciona; `toSorted()` devolve uma cópia ordenada sem alterar o estado original. Se o ambiente não oferecer `toSorted()`, use `[...registros].sort(...)`.

12.4 Estados de carregamento, vazio, erro e sucesso

Uma interface não possui apenas o estado “com dados”. Ela deve explicar o que ocorre durante a espera, quando a busca não encontra resultados, quando uma operação falha e quando termina corretamente.

JAVASCRIPT

```
1  async function atualizarListagem() {  
2      status.textContent = 'Carregando registros...'  
3      tabela.hidden = true  
4  
5      try {  
6          const registros = await listarRegistros()  
7          renderizarRegistros(registros)  
8          tabela.hidden = registros.length === 0  
9          status.textContent = registros.length ? '' : 'Nenhum registro encontrado.'  
10     } catch (erro) {  
11         status.textContent = 'Não foi possível carregar os registros.'  
12     }  
13 }
```

O elemento de status pode usar `aria-live='polite'` para anunciar mudanças importantes. Preserva a mensagem enquanto ela for útil e oferece uma ação de nova tentativa quando a falha puder ser temporária.

PERGUNTA: Por que `value` não é o próprio texto digitado?

RESPOSTA Porque o campo selecionado é um objeto DOM. A variável guarda o elemento; sua propriedade `value` guarda a string atual. Em um campo numérico, `valueAsNumber` oferece o número ou `NaN`.

13 Tempo, callbacks e promessas

JavaScript executa uma pilha de chamadas por vez. Operações como temporizadores e rede são coordenadas pelo ambiente; quando terminam, suas continuações entram em filas para execução futura. Assíncrono não significa “duas linhas JavaScript executando ao mesmo tempo”.

13.1 Temporizadores e ordem de execução

JAVASCRIPT

```
1 console.log('A')
2
3 setTimeout(() => {
4   console.log('B')
5 }, 0)
6
7 console.log('C')
```

SAÍDA

```
A
C
B
```

O callback do temporizador só executa depois que a pilha atual termina. O zero indica atraso mínimo, não execução imediata.

13.2 Promise e seus estados

Uma promessa começa pendente e termina cumprida ou rejeitada. `then` trata sucesso; `catch`, falha; `finally`, encerramento.

JAVASCRIPT

```
1 function esperar(ms) {
2   return new Promise(resolve => setTimeout(resolve, ms))
3 }
4
5 esperar(500)
6   .then(() => console.log('meio segundo depois'))
7   .catch(erro => console.error(erro))
```

13.3 async e await

`async` faz a função devolver uma Promise. `await` pausa somente aquela função até a promessa ser resolvida.

JAVASCRIPT

```
1 async function executar() {  
2   try {  
3     await esperar(500)  
4     return 'concluído'  
5   } catch (erro) {  
6     return `falhou: ${erro.message}`  
7   }  
8 }  
9  
10 executar().then(console.log)
```

SAÍDA

concluído

14 HTTP, fetch e JSON

Uma requisição HTTP combina método, URL, cabeçalhos e, quando necessário, corpo. A resposta possui status, cabeçalhos e corpo. JSON é um formato frequente para o corpo, não o protocolo inteiro.

14.1 Ler dados com GET

JAVASCRIPT

```
1 async function carregarRegistro(id) {
2   const resposta = await fetch(`/api/registros/${id}`)
3
4   if (!resposta.ok) {
5     throw new Error(`HTTP ${resposta.status}`)
6   }
7
8   return resposta.json()
9 }
10
11 carregarRegistro(1)
12   .then(registro => console.log(registro.titulo))
13   .catch(erro => console.error(erro.message))
```

FETCH NÃO REJEITA EM TODO ERRO HTTP

Uma resposta 404 ou 500 normalmente resolve a Promise. Verifique `resposta.ok` ou `resposta.status`. A Promise costuma rejeitar em falhas de rede, abortos ou políticas do navegador.

14.2 Busca, filtros, ordenação e paginação na URL

Consultas que não alteram dados usam parâmetros na URL. `URLSearchParams` aplica a codificação correta e evita concatenar manualmente espaços, acentos e símbolos.

JAVASCRIPT

```
1 async function listarRegistros(filtros) {
2   const parametros = new URLSearchParams()
3
4   if (filtros.texto) parametros.set('q', filtros.texto.trim())
5   if (filtros.situacao) parametros.set('situacao', filtros.situacao)
6   parametros.set('ordenarPor', filtros.ordenarPor ?? 'atualizacao')
7   parametros.set('pagina', String(filtros.pagina ?? 1))
8   parametros.set('limite', String(filtros.limite ?? 20))
9
10  const resposta = await fetch(`/api/registros?${parametros}`)
11  if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`)
12  return resposta.json()
13 }
```

O servidor continua responsável por validar os critérios, limitar tamanhos e escolher as colunas permitidas para ordenação. A resposta pode trazer `itens`, `pagina`, `limite` e `total` para a interface montar a paginação.

14.3 Enviar JSON com POST

JAVASCRIPT

```
1 async function criarRegistro(registro) {
2   const resposta = await fetch('/api/registros', {
3     method: 'POST',
4     headers: {
5       'Content-Type': 'application/json'
6     },
7     body: JSON.stringify(registro)
8   })
9
10  if (!resposta.ok) throw new Error('Não foi possível criar o registro.')
11  return resposta.json()
12 }
```

JSON

```
1 {
2   "titulo": "Revisar documentação",
3   "prioridade": "MEDIA"
4 }
```

Dados estruturados são enviados no corpo. Cabeçalhos descrevem metadados, como o tipo do conteúdo; não são o lugar usual para os dados do formulário.

14.4 Atualização parcial e cancelamento lógico

PATCH comunica uma alteração parcial. Quando um registro precisa permanecer para auditoria ou histórico, prefira alterar seu estado em vez de apagá-lo fisicamente.

JAVASCRIPT

```
1 async function alterarSituacao(id, situacao) {
2   const resposta = await fetch(`/api/registros/${id}/situacao`, {
3     method: 'PATCH',
4     headers: { 'Content-Type': 'application/json' },
5     body: JSON.stringify({ situacao })
6   })
7
8   if (!resposta.ok) {
9     const erro = await resposta.json().catch(() => ({}))
10    throw new Error(erro.mensagem ?? 'Alteração recusada pela API.')
11  }
12
13  return resposta.json()
14 }
```

A interface não decide sozinha se uma transição é permitida. Ela pode ocultar ou desabilitar ações para orientar a pessoa, mas o servidor deve conferir novamente estado atual, identidade e permissão.

14.5 Métodos e status essenciais

Método	Intenção comum
GET	Consultar uma representação sem alterar o recurso.
POST	Criar ou disparar um processamento com dados no corpo.
PUT	Substituir uma representação completa.
PATCH	Alterar parcialmente.
DELETE	Remover.

14.6 Um cliente HTTP reutilizável

Centralizar a leitura de respostas reduz repetição e torna as mensagens consistentes. Também é preciso lidar com respostas sem corpo, como 204 `No Content`.

JAVASCRIPT

```
1 async function requisitar(url, opcoes = {}) {
2   const resposta = await fetch(url, opcoes)
3   const tipo = resposta.headers.get('content-type') ?? ''
4   const possuiJson = tipo.includes('application/json')
5   const corpo = possuiJson ? await resposta.json() : null
6
7   if (!resposta.ok) {
8     const mensagem = corpo?.mensagem ?? `Falha HTTP ${resposta.status}`
9     throw new Error(mensagem)
10  }
11
12  return corpo
13 }
```

Em aplicações autenticadas, o contrato pode exigir cookie de sessão ou cabeçalho de autorização. Não espalhe essa decisão por todas as telas: configure-a no cliente HTTP e nunca registre credenciais no console.

14.7 Abortar requisições e evitar resultados antigos

Ao pesquisar enquanto a pessoa digita, uma resposta antiga pode chegar depois da nova. `AbortController` permite cancelar a solicitação anterior.

JAVASCRIPT

```
1 let controladorAtual
2
3 async function pesquisar(termo) {
4   controladorAtual?.abort()
5   controladorAtual = new AbortController()
6
7   try {
```

```

8     return await requisitar(`/api/registros?q=${encodeURIComponent(termo)}`, {
9       signal: controladorAtual.signal
10    })
11  } catch (erro) {
12    if (erro.name === 'AbortError') return null
13    throw erro
14  }
15 }

```

Para campos de busca, combine o cancelamento com um pequeno *debounce*: espere alguns milissegundos após a última digitação antes de consultar a API.

14.8 Consumir um serviço externo

Serviços externos podem estar lentos, indisponíveis ou devolver um formato inesperado. Isole a integração em uma função, valide o resultado e transforme-o no formato interno da aplicação.

JAVASCRIPT

```

1  async function consultarData(dataISO) {
2    const resposta = await fetch(`/api/calendario?data=${encodeURIComponent(dataISO)}`)
3    if (!resposta.ok) throw new Error('Não foi possível verificar a data.')
4
5    const dados = await resposta.json()
6    if (typeof dados.indisponivel !== 'boolean') {
7      throw new Error('Resposta de calendário inválida.')
8    }
9
10   return dados.indisponivel
11 }

```

Quando a verificação define uma regra importante, o backend deve realizá-la mesmo que o frontend já tenha consultado o serviço. O servidor também pode esconder chaves privadas, aplicar timeout e controlar cache.

Faixa	Interpretação
2xx	Sucesso: 200 OK, 201 Created, 204 No Content.
4xx	Problema na requisição: 400, 404, 409, 422.
5xx	Falha no servidor: 500 e variantes.

14.9 Concorrência com Promise.all

JAVASCRIPT

```

1  const [curso, alunos] = await Promise.all([
2    fetch('/api/curso').then(r => r.json()),
3    fetch('/api/alunos').then(r => r.json())
4  ])

```

Quando operações são independentes, iniciá-las juntas reduz o tempo total. Se uma rejeitar,

`Promise.all` rejeita; use `Promise.allSettled` quando quiser observar todos os resultados.

15 Módulos e organização

Módulos evitam variáveis globais e tornam dependências explícitas. Cada arquivo exporta uma interface pequena e importa somente o necessário.

15.1 export e import

JAVASCRIPT

```
1 // matematica.js
2 export function somar(a, b) {
3   return a + b
4 }
5
6 export const PI = 3.14159
```

JAVASCRIPT

```
1 // app.js
2 import { somar, PI } from './matematica.js'
3
4 console.log(somar(2, 3), PI)
```

No navegador:

HTML

```
1 <script type="module" src="app.js"></script>
```

Módulos usam modo estrito, têm escopo próprio e obedecem às políticas de origem. Abra o projeto por um servidor local, não apenas por `file://`, quando houver importações ou requisições.

15.2 Um desenho por camadas

JAVASCRIPT

```
1 // dominio/transicoes.js: regra pura
2 const permitidas = {
3   NOVO: ['EM_ANDAMENTO', 'CANCELADO'],
4   EM_ANDAMENTO: ['EM_REVISAO', 'CANCELADO'],
5   EM_REVISAO: ['EM_ANDAMENTO', 'CONCLUIDO', 'CANCELADO']
6 }
7
8 export function podeAlterarSituacao(atual, proxima) {
9   return permitidas[atual]?.includes(proxima) ?? false
10 }
11
```

```
12 // ui/app.js: DOM e eventos
13 // api/cliente.js: fetch e HTTP
```

Regra de domínio não depende do DOM nem da rede; por isso, pode ser reutilizada no navegador, no servidor e nos testes. `await` suspende só a função `async`, não a página.

PARTE III

Ecossistema e aplicação

16 Node.js, npm e APIs com Express

Node.js é um ambiente de execução de JavaScript fora do navegador. Ele usa o motor V8 e oferece APIs para arquivos, processos, rede e sistema operacional. Não inclui o DOM: não existe `document.querySelector()` em um processo Node comum.

16.1 package.json, node_modules e lockfile

Um diretório não se torna uma aplicação Node por conter qualquer arquivo `.js`. O manifesto `package.json` descreve nome, scripts, dependências e configuração do pacote.

TERMINAL

```
mkdir painel-api
cd painel-api
npm init -y
npm install express cors
```

Item	Papel
<code>package.json</code>	Manifesto do projeto e dependências declaradas.
<code>package-lock.json</code>	Versões resolvidas para instalações reproduzíveis.
<code>node_modules/</code>	Pacotes instalados localmente; pode ser recriado com <code>npm install</code> .
<code>.gitignore</code>	Lista o que não deve ser versionado; geralmente inclui <code>node_modules/</code> .

NÃO EXISTE "PORTA PADRÃO DO NODE"

`3000` é uma convenção comum em exemplos, não uma porta obrigatória do Node.js. O programa escolhe a porta, frequentemente por `process.env.PORT ?? 3000`.

16.2 Scripts npm e módulos

JSON

```
1 {
2   "name": "painel-api",
3   "version": "1.0.0",
4   "type": "module",
5   "scripts": {
6     "dev": "node --watch src/server.js",
7     "start": "node src/server.js"
8   },
9   "dependencies": {
10    "express": "^5.0.0"
11  }
12 }
```

TERMINAL

```
npm run dev
```

O script dá um nome estável ao comando. O campo "type": "module" habilita `import/export` em arquivos `.js` daquele pacote.

16.3 Servidor HTTP com Express

JAVASCRIPT

```
1 import express from 'express'
2
3 const app = express()
4 app.use(express.json())
5
6 app.get('/health', (req, res) => {
7   res.json({ status: 'ok' })
8 })
9
10 const porta = process.env.PORT ?? 3000
11 app.listen(porta, () => {
12   console.log(`Servidor em http://localhost:${porta}`)
13 })
```

RESPOSTA DE GET /health

```
200 OK
{"status":"ok"}
```

`express.json()` lê corpos JSON e disponibiliza o valor em `req.body`. Rotas combinam método e caminho. `res.status(...).json(...)` define status e serializa a resposta.

16.4 Validar entrada de uma rota

REQUISIÇÃO E RESPOSTA**POST /somar**

Corpo enviado: `{"a":2,"b":3}`

Resposta: 200 OK – `{"resultado":5}`

JAVASCRIPT

```

1 app.post('/somar', (req, res) => {
2   const { a, b } = req.body
3
4   if (!Number.isFinite(a) || !Number.isFinite(b)) {
5     return res.status(400).json({ erro: 'a e b devem ser números' })
6   }
7
8   return res.status(200).json({ resultado: a + b })
9 })

```

Validação no cliente melhora a experiência; validação no servidor protege a regra. Nunca confie que a requisição veio da sua página.

16.5 Rotas, middlewares e camadas

Uma rota deve coordenar HTTP, não concentrar consulta ao banco, regra de negócio, autenticação e formatação. Uma separação simples torna cada responsabilidade localizável.

ESTRUTURA POSSÍVEL

```

1 src/
2 |-- app.ts
3 |-- server.ts
4 |-- routes/
5 |   |-- registros.routes.ts
6 |-- controllers/
7 |   |-- registros.controller.ts
8 |-- services/
9 |   |-- registros.service.ts
10 |-- repositories/
11 |   |-- registros.repository.ts
12 |-- middlewares/
13 |   |-- autenticacao.ts
14 |   |-- erros.ts
15 |-- types/
16   |-- registro.ts

```

Camada	Responsabilidade principal
Rota	Combinar método, caminho e sequência de middlewares.
Controller	Ler a requisição, chamar o serviço e construir a resposta HTTP.
Service	Aplicar regras, transições, permissões e coordenação de operações.
Repository	Isolar consultas e comandos do mecanismo de persistência.
Middleware	Executar preocupação transversal, como autenticação ou tratamento de erro.

TYPESCRIPT

```

1  const router = Router()
2
3  router.get('/', registrosController.listar)
4  router.get('/:id', registrosController.detalhar)
5  router.post('/', exigirAutenticacao, registrosController.criar)
6  router.patch('/:id', exigirAutenticacao, registrosController.atualizar)
7  router.patch('/:id/situacao', exigirAutenticacao,
8    registrosController.alterarSituacao)

```

Router funciona como uma pequena aplicação de rotas. A ordem dos middlewares importa: cada um encerra a resposta ou chama `next()` para passar o controle.

16.6 Persistência, histórico e transações

O repositório deve usar parâmetros oferecidos pelo driver, em vez de concatenar entrada em SQL. Quando uma operação altera o registro principal e também cria histórico, as duas escritas formam uma única unidade: ou ambas confirmam, ou ambas são desfeitas.

TYPESCRIPT

```

1  async function alterarSituacao(comando: AlterarSituacao) {
2    return banco.transacao(async conexao => {
3      const atual = await repositorio.buscarPorId(conexao, comando.id)
4      if (!atual) throw new ErroHttp(404, 'Registro não encontrado.')
5
6      autorizarAlteracao(comando.usuario, atual, comando.proxima)
7      validarTransicao(atual.situacao, comando.proxima)
8
9      await repositorio.atualizarSituacao(conexao, comando.id, comando.proxima)
10     await historicoRepositorio.adicionar(conexao, {
11       registroId: comando.id,
12       usuarioId: comando.usuario.id,
13       descricao: `${atual.situacao} -> ${comando.proxima}`
14     })
15   })
16 }

```

O exemplo mostra o contrato entre camadas; os nomes concretos de transação dependem do driver. Datas de criação e atualização devem ser definidas no servidor ou no banco. Para preservar auditoria, uma operação de cancelamento pode atualizar estado e manter o registro, em vez de usar exclusão física.

16.7 Parâmetros de rota e de consulta

`req.params` contém partes do caminho, como o identificador; `req.query` contém busca, filtros, ordenação e paginação. Tudo chega como dado externo e precisa ser validado.

TYPESCRIPT

```

1  app.get('/api/registros/:id', async (req, res) => {
2    const id = Number(req.params.id)
3    if (!Number.isInteger(id) || id <= 0) {

```

```
4     return res.status(400).json({ mensagem: 'Identificador inválido.' })
5   }
6
7   const registro = await servico.buscarPorId(id)
8   if (!registro) return res.status(404).json({ mensagem: 'Não encontrado.' })
9   return res.json(registro)
10 }
```

Para ordenação, compare o nome solicitado com uma lista permitida antes de montar a consulta. Nunca use diretamente uma string externa como nome de coluna, trecho SQL ou caminho de arquivo.

16.8 Operação segura e integração

Resumos para painéis

Indicadores são dados derivados: totais por situação, tipo, prioridade ou intervalo de data. Para bases grandes, o servidor deve pedir agregações ao banco em vez de carregar todos os registros e contar no navegador.

RESPOSTA DE GET /api/resumo

```
{"total":128,"porSituacao":{"NOVO":21,"CONCLUIDO":54}, "atencao":7}
```

Mantenha nomes e significados estáveis no contrato. Um valor como `atencao` precisa ter uma regra documentada; por exemplo, registros críticos ainda não concluídos. A interface apenas apresenta o resultado calculado pela camada adequada.

Integração externa com timeout

Uma API externa é outra entrada não confiável. Valide a data antes da chamada, aplique timeout, confira o status e converta a resposta para um tipo interno pequeno.

TYPESCRIPT

```
1 async function consultarCalendario(dataISO: string): Promise<boolean> {
2   if (!/^\d{4}-\d{2}-\d{2}$/.test(dataISO)) {
3     throw new ErroHttp(400, 'Data inválida.')
4   }
5
6   const base = process.env.CALENDARIO_URL
7   if (!base) throw new Error('CALENDARIO_URL não configurada')
8
9   const url = new URL('/datas', base)
10  url.searchParams.set('data', dataISO)
11
12  const resposta = await fetch(url, {
13    signal: AbortSignal.timeout(4000)
14  })
15  if (!resposta.ok) throw new ErroHttp(502, 'Serviço externo indisponível.')
16
17  const dados: unknown = await resposta.json()
18  if (!ehRespostaCalendario(dados)) {
```

```
19     throw new ErroHttp(502, 'Resposta externa inválida.')
20   }
21
22   return dados indisponível
23 }
```

Uma chave privada permanece em variável de ambiente no backend. Defina também o comportamento quando o serviço falhar: bloquear a operação, permitir com aviso ou tentar novamente é uma decisão do requisito, não do `fetch`.

Configuração, CORS e segredos

Configurações que mudam entre ambientes devem vir de `process.env`. Um arquivo local `.env` pode facilitar o desenvolvimento, mas não deve ser versionado quando contém segredo.

TYPESCRIPT

```
1 const porta = Number(process.env.PORT ?? 3000)
2 const origemPermitida = process.env.FRONTEND_ORIGIN
3
4 if (!origemPermitida) {
5   throw new Error('FRONTEND_ORIGIN não configurada')
6 }
7
8 app.use(cors({
9   origin: origemPermitida,
10  credentials: true
11 })))
```

CORS controla quais origens podem ler respostas no navegador; não autentica usuários e não substitui autorização. Se frontend e API compartilham a mesma origem, talvez não seja necessário habilitá-lo.

Autenticação e autorização

Autenticação responde “quem fez a requisição?”. Autorização responde “essa identidade pode executar esta operação?”. A interface pode adaptar menus e botões, mas a decisão definitiva pertence ao servidor.

TYPESCRIPT

```
1 type Perfil = 'ADMIN' | 'GESTOR' | 'MEMBRO'
2
3 function permitir(...perfis: Perfil[]) {
4   return (req: RequestAutenticada, res: Response, next: NextFunction) => {
5     if (!req.usuario) {
6       return res.status(401).json({ mensagem: 'Autenticação necessária.' })
7     }
8
9     if (!perfis.includes(req.usuario.perfil)) {
10      return res.status(403).json({ mensagem: 'Operação não permitida.' })
11    }
12
13    next()
14  }
```

```

14   }
15 }
16
17 router.patch('/:id/situacao', permitir('ADMIN', 'GESTOR'), alterarSituacao)

```

401 indica ausência ou falha de autenticação; 403, identidade conhecida sem permissão. Senhas devem ser comparadas por meio de hash produzido por biblioteca apropriada. Nunca armazene senha em texto simples nem implemente criptografia própria.

OCULTAR BOTÃO NÃO PROTEGE A OPERAÇÃO

Uma pessoa pode chamar a API sem usar sua página. Confira no servidor a identidade, o perfil, a associação ao recurso e a transição solicitada em todas as rotas protegidas.

Sessão e login no navegador

O formulário de login envia credenciais uma vez para um endpoint protegido por HTTPS. O servidor compara o hash da senha e cria uma sessão. Quando a sessão usa cookie `HttpOnly`, o JavaScript não lê o segredo; o navegador o envia conforme a política configurada.

JAVASCRIPT

```

1  async function entrar(email, senha) {
2    const resposta = await fetch('/api/sessoes', {
3      method: 'POST',
4      credentials: 'include',
5      headers: { 'Content-Type': 'application/json' },
6      body: JSON.stringify({ email, senha })
7    })
8
9    if (!resposta.ok) throw new Error('E-mail ou senha inválidos.')
10   return resposta.json()
11 }
12
13 async function sair() {
14   await fetch('/api/sessoes/atual', {
15     method: 'DELETE',
16     credentials: 'include'
17   })
18 }

```

Não persista a senha. Evite guardar tokens sensíveis em `localStorage`; uma falha de injeção de script poderia lê-los. Cookies de sessão exigem configuração segura, política `SameSite` e proteção contra CSRF conforme a arquitetura adotada.

Tratamento centralizado de erros

Erros conhecidos podem carregar status e mensagem pública. Falhas inesperadas devem ser registradas no servidor, enquanto a resposta evita expor pilha, SQL, caminhos ou segredos.

TYPESCRIPT

```
1 app.use((erro: unknown, req: Request, res: Response, next: NextFunction) => {  
2   console.error(erro)  
3  
4   if (erro instanceof ErrorHttp) {  
5     return res.status(erro.status).json({ mensagem: erro.message })  
6   }  
7  
8   return res.status(500).json({ mensagem: 'Erro interno do servidor.' })  
9 })
```

O middleware de erro possui quatro parâmetros e deve ser registrado depois das rotas. Em Express 5, rejeições de handlers assíncronos são encaminhadas ao tratamento de erro; ainda assim, escreva serviços que lancem erros claros e evite rejeições esquecidas.

17 TypeScript sem mistério

TypeScript é JavaScript com uma camada de tipos estáticos e ferramentas. O navegador e o Node executam JavaScript; o compilador remove as anotações de tipo e produz o arquivo executável.

17.1 Tipos básicos e inferência

TYPESCRIPT

```
1 let total: number = 0
2 const curso = 'JavaScript' // inferido como string
3
4 function somar(a: number, b: number): number {
5   return a + b
6 }
```

Tipos ajudam antes da execução. Eles não validam automaticamente dados que chegaram da rede; esses valores ainda precisam de checagem em tempo de execução.

17.2 Objetos, aliases e união

TYPESCRIPT

```
1 type Situacao = 'NOVO' | 'EM_ANDAMENTO' | 'EM_REVISAO' | 'CONCLUIDO' | 'CANCELADO'
2 type Prioridade = 'BAIXA' | 'MEDIA' | 'ALTA' | 'CRITICA'
3
4 type Registro = {
5   id: number
6   titulo: string
7   situacao: Situacao
8   prioridade: Prioridade
9   responsavelId: number | null
10  prazo: string | null
11 }
12
13 function descrever(registro: Registro): string {
14   return `${registro.titulo} - ${registro.situacao}`
15 }
```

A união restringe o conjunto de strings válidas. A interrogação torna uma propriedade opcional; seu valor pode ser `undefined`.

17.3 Narrowing e unknown

TYPESCRIPT

```

1 function paraNumero(valor: unknown): number | null {
2   if (typeof valor === 'number' && Number.isFinite(valor)) {
3     return valor
4   }
5
6   if (typeof valor === 'string' && valor.trim() !== '') {
7     const convertido = Number(valor)
8     return Number.isFinite(convertido) ? convertido : null
9   }
10
11   return null
12 }

```

`unknown` obriga a investigar o valor antes de usá-lo. `any` desliga essa proteção e deve ser exceção, não atalho rotineiro.

17.4 Tipos internos e validação de entrada

Uma anotação não transforma `req.body` em dado confiável. Primeiro valide o valor desconhecido; somente depois produza um objeto interno tipado. Bibliotecas de esquema podem reduzir repetição, mas o princípio permanece o mesmo.

TYPESCRIPT

```

1 type CriarRegistro = {
2   titulo: string
3   prioridade: Prioridade
4   prazo: string | null
5 }
6
7 function lerCriacao(entrada: unknown): CriarRegistro | null {
8   if (typeof entrada !== 'object' || entrada === null) return null
9
10  const dados = entrada as Record<string, unknown>
11  const prioridades: Prioridade[] = ['BAIXA', 'MEDIA', 'ALTA', 'CRITICA']
12
13  if (typeof dados.titulo !== 'string' || dados.titulo.trim().length < 3) {
14    return null
15  }
16  if (!prioridades.includes(dados.prioridade as Prioridade)) return null
17  if (dados.prazo !== null && typeof dados.prazo !== 'string') return null
18
19  return {
20    titulo: dados.titulo.trim(),
21    prioridade: dados.prioridade as Prioridade,
22    prazo: dados.prazo as string | null
23  }
24 }

```

Separe tipos de criação, atualização, resposta pública e registro persistido. Um campo gerado pelo servidor, como `id` ou data de criação, não precisa ser aceito no corpo de cadastro.

17.5 Configuração estrita

Ative as verificações estritas desde o início. Adiar essa decisão faz o compilador aceitar suposições que depois se espalham pelo código.

JSON

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "NodeNext",
5     "moduleResolution": "NodeNext",
6     "rootDir": "src",
7     "outDir": "dist",
8     "strict": true,
9     "noUncheckedIndexedAccess": true,
10    "sourceMap": true
11  },
12  "include": ["src"]
13 }
```

`strict` reúne várias verificações importantes. `noUncheckedIndexedAccess` lembra que um índice pode não existir. Ajuste alvo e módulos ao ambiente realmente adotado, sem copiar configurações incompatíveis.

Para executar TypeScript durante o desenvolvimento e gerar JavaScript para a execução final:

TERMINAL

```
npm install -D typescript tsx @types/node @types/express @types/cors
npm run dev
npm run build
npm start
```

JSON

```
1 {
2   "scripts": {
3     "dev": "tsx watch src/server.ts",
4     "build": "tsc",
5     "start": "node dist/server.js"
6   }
7 }
```

`tsx` executa a fonte no desenvolvimento; `tsc` verifica e compila; o script `start` executa somente a saída JavaScript. O `package.json` real também mantém dependências e demais metadados.

17.6 Módulos e compilação

TYPESCRIPT

```
1 // src/domain/transicoes.ts
2 export function podeAlterarSituacao(
3   atual: Situacao,
4   proxima: Situacao
5 ): boolean {
6   const permitidas: Partial<Record<Situacao, Situacao[]>> = {
7     NOVO: ['EM_ANDAMENTO', 'CANCELADO'],
8     EM_ANDAMENTO: ['EM_REVISAO', 'CANCELADO'],
9     EM_REVISAO: ['EM_ANDAMENTO', 'CONCLUIDO', 'CANCELADO']
10  }
11
12   return permitidas[atual]?.includes(proxima) ?? false
13 }
```

TYPESCRIPT

```
1 // src/services/registros.service.ts
2 import { podeAlterarSituacao } from '../domain/transicoes.js'
3
4 if (!podeAlterarSituacao(registro.situacao, proxima)) {
5   throw new ErroHttp(409, 'Transição de situação não permitida.')
6 }
```

O caminho `.js` pode ser necessário na saída ESM mesmo quando a fonte é `.ts`. A configuração exata depende do `tsconfig.json` e do ambiente.

TYPESCRIPT NÃO CORRIGE A LÓGICA

O compilador pode confirmar que dois estados pertencem à união `Situacao`, mas não sabe, por si só, se uma transição atende à regra. Tipos complementam validação e testes; não os substituem.

18 Bootstrap e JavaScript de componentes

Bootstrap é um framework de front-end. Seu CSS fornece grid, utilitários e componentes visuais; seu JavaScript implementa comportamentos como modal, dropdown e collapse. Ele não fornece backend nem banco de dados.

18.1 Arquivos minificados

Arquivos `.min.css` e `.min.js` removem espaços, comentários e outras partes desnecessárias para reduzir transferência. A minificação não existe para tornar o código mais legível; use fontes normais no desenvolvimento quando precisar depurar e versões minificadas na entrega.

HTML

```
1 <link rel="stylesheet" href="bootstrap.min.css">
2
3 <script src="bootstrap.bundle.min.js"></script>
```

O `bundle` inclui a dependência Popper usada por componentes que posicionam elementos. Os nomes e formas de distribuição podem variar entre versões; consulte a documentação da versão instalada.

18.2 Data attributes e API JavaScript

HTML

```
1 <button class="btn btn-primary"
2       data-bs-toggle="collapse"
3       data-bs-target="#detalhes">
4   Mostrar detalhes
5 </button>
6
7 <div class="collapse" id="detalhes">
8   Conteúdo revelado pelo componente Collapse.
9 </div>
```

Bootstrap encontra os atributos `data-bs-*` e registra o comportamento. Também é possível controlar componentes pela API JavaScript quando o estado depende da aplicação.

PERGUNTA: TypeScript substitui JavaScript?

RESPOSTA Não. TypeScript descreve e verifica o código durante o desenvolvimento; depois gera JavaScript. A semântica de execução continua sendo a de JavaScript.

19 Exemplo integrado: painel de registros

Este exemplo genérico reúne formulário, listagem, busca, filtro, estados de interface, módulos, `fetch`, API com Express e tipos compartilhados. O armazenamento em memória mantém o foco no JavaScript; em uma aplicação real, o repositório seria substituído por persistência relacional.

19.1 Estrutura

ESTRUTURA

```
1 painel/  
2 |-- package.json  
3 |-- tsconfig.json  
4 |-- public/  
5 |   |-- index.html  
6 |   |-- app.js  
7 |-- src/  
8     |-- registro.ts  
9     |-- server.ts
```

19.2 Modelo e regras

TYPESCRIPT

```
1 // src/registro.ts  
2 export type Situacao = 'ATIVO' | 'CANCELADO'  
3 export type Prioridade = 'BAIXA' | 'MEDIA' | 'ALTA'  
4  
5 export type Registro = {  
6   id: number  
7   titulo: string  
8   categoria: string  
9   prioridade: Prioridade  
10  situacao: Situacao  
11  criadoEm: string  
12 }  
13  
14 export function validarTitulo(titulo: unknown): string {  
15   if (typeof titulo !== 'string' || titulo.trim().length < 3) {  
16     throw new Error('0 título deve possuir ao menos três caracteres.')  
17   }  
18   return titulo.trim()  
19 }
```

Tipos documentam o estado interno; a função ainda valida o valor recebido em tempo de execução. O identificador, a situação inicial e a data são definidos pelo servidor, não aceitos como decisão do cliente.

19.3 Interface HTML

HTML

```

1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Painel de registros</title>
7   <script type="module" src="app.js"></script>
8 </head>
9 <body>
10  <main>
11    <h1>Painel de registros</h1>
12
13    <form id="cadastro-form">
14      <label for="titulo">Título</label>
15      <input id="titulo" name="titulo" minlength="3" required>
16
17      <label for="categoria">Categoria</label>
18      <input id="categoria" name="categoria" required>
19
20      <label for="prioridade">Prioridade</label>
21      <select id="prioridade" name="prioridade" required>
22        <option value="MEDIA">Média</option>
23        <option value="BAIXA">Baixa</option>
24        <option value="ALTA">Alta</option>
25      </select>
26      <button type="submit">Cadastrar</button>
27    </form>
28
29    <form id="filtros-form" role="search">
30      <label for="busca">Buscar</label>
31      <input id="busca" name="q" type="search">
32      <label for="situacao">Situação</label>
33      <select id="situacao" name="situacao">
34        <option value="">Todas</option>
35        <option value="ATIVO">Ativo</option>
36        <option value="CANCELADO">Cancelado</option>
37      </select>
38      <button type="submit">Filtrar</button>
39    </form>
40
41    <p id="status" role="status" aria-live="polite"></p>
42    <table id="tabela">
43      <caption>Registros encontrados</caption>
44      <thead><tr>
45        <th scope="col">Título</th><th scope="col">Prioridade</th>
46        <th scope="col">Situação</th><th scope="col">Ação</th>
47      </tr></thead>
48      <tbody id="registros"></tbody>
49    </table>
50  </main>
51 </body>
52 </html>

```

Os dois formulários representam intenções diferentes: cadastro altera dados; busca apenas consulta. O elemento de status comunica carregamento, ausência de resultados, erro e sucesso.

19.4 Cliente HTTP e renderização

JAVASCRIPT

```

1 // public/app.js
2 const cadastro = document.querySelector('#cadastro-form')
3 const filtros = document.querySelector('#filtros-form')
4 const corpo = document.querySelector('#registros')
5 const tabela = document.querySelector('#tabela')
6 const status = document.querySelector('#status')
7
8 async function requisitar(url, opcoes) {
9   const resposta = await fetch(url, opcoes)
10  const dados = await resposta.json().catch(() => null)
11  if (!resposta.ok) {
12    throw new Error(dados?.mensagem ?? `Falha HTTP ${resposta.status}`)
13  }
14  return dados
15 }
16
17 function célula(texto) {
18   const td = document.createElement('td')
19   td.textContent = texto
20   return td
21 }
22
23 function renderizar(registros) {
24   const fragmento = document.createDocumentFragment()
25
26   for (const registro of registros) {
27     const linha = document.createElement('tr')
28     linha.dataset.id = registro.id
29
30     const acao = document.createElement('button')
31     acao.type = 'button'
32     acao.dataset.acao = 'cancelar'
33     acao.textContent = 'Cancelar'
34     acao.disabled = registro.situacao === 'CANCELADO'
35
36     const célulaAcao = document.createElement('td')
37     célulaAcao.append(acao)
38     linha.append(
39       célula(registro.titulo), célula(registro.prioridade),
40       célula(registro.situacao), célulaAcao
41     )
42     fragmento.append(linha)
43   }
44
45   corpo.replaceChildren(fragmento)
46   tabela.hidden = registros.length === 0
47   status.textContent = registros.length ? '' : 'Nenhum registro encontrado.'
48 }

```

19.5 Carregar, cadastrar e cancelar

JAVASCRIPT

```

1  async function carregar() {
2    status.textContent = 'Carregando...'
3    tabela.hidden = true
4
5    try {
6      const parametros = new URLSearchParams(new FormData(filtros))
7      const dados = await requisitar(`/api/registros?${parametros}`)
8      renderizar(dados.itens)
9    } catch (erro) {
10     status.textContent = erro.message
11   }
12 }
13
14 filtros.addEventListener('submit', event => {
15   event.preventDefault()
16   carregar()
17 })
18
19 cadastro.addEventListener('submit', async event => {
20   event.preventDefault()
21   const botao = cadastro.querySelector('button[type="submit"]')
22   botao.disabled = true
23
24   try {
25     const dados = Object.fromEntries(new FormData(cadastro))
26     await requisitar('/api/registros', {
27       method: 'POST',
28       headers: { 'Content-Type': 'application/json' },
29       body: JSON.stringify(dados)
30     })
31     cadastro.reset()
32     status.textContent = 'Registro criado.'
33     await carregar()
34   } catch (erro) {
35     status.textContent = erro.message
36   } finally {
37     botao.disabled = false
38   }
39 })

```

JAVASCRIPT

```

1  corpo.addEventListener('click', async event => {
2    const botao = event.target.closest('[data-acao="cancelar"]')
3    if (!botao) return
4
5    const linha = botao.closest('tr')
6    botao.disabled = true
7    try {
8      await requisitar(`/api/registros/${linha.dataset.id}/situacao`, {
9        method: 'PATCH',
10       headers: { 'Content-Type': 'application/json' },
11       body: JSON.stringify({ situacao: 'CANCELADO' })
12     })
13     await carregar()
14   } catch (erro) {

```

```

15     status.textContent = erro.message
16     botao.disabled = false
17   }
18 })
19
20 carregar()

```

A delegação permite tratar todos os botões atuais e futuros com um listener. Desabilitar durante a requisição reduz envios duplicados; a API continua responsável por validar a operação.

19.6 API Express em TypeScript

TYPESCRIPT

```

1  // src/server.ts
2  import express from 'express'
3  import type { Prioridade, Registro } from './registro.js'
4  import { validarTitulo } from './registro.js'
5
6  const app = express()
7  app.use(express.json())
8  app.use(express.static('public'))
9
10 let proximoId = 2
11 const registros: Registro[] = [{
12   id: 1, titulo: 'Preparar material', categoria: 'Conteúdo',
13   prioridade: 'MEDIA', situacao: 'ATIVO',
14   criadoEm: new Date().toISOString()
15 }]
16
17 app.get('/api/registros', (req, res) => {
18   const q = typeof req.query.q === 'string' ? req.query.q.trim().toLowerCase() : ''
19   const situacao = typeof req.query.situacao === 'string' ? req.query.situacao : ''
20
21   const itens = registros.filter(item =>
22     (!q || item.titulo.toLowerCase().includes(q)) &&
23     (!situacao || item.situacao === situacao)
24   )
25   res.json({ itens, total: itens.length })
26 })

```

TYPESCRIPT

```

1  app.post('/api/registros', (req, res) => {
2    try {
3      const titulo = validarTitulo(req.body?.titulo)
4      const prioridades: Prioridade[] = ['BAIXA', 'MEDIA', 'ALTA']
5      if (!prioridades.includes(req.body?.prioridade)) {
6        return res.status(400).json({ mensagem: 'Prioridade inválida.' })
7      }
8
9      const registro: Registro = {
10        id: proximoId++, titulo,
11        categoria: String(req.body?.categoria ?? '').trim(),
12        prioridade: req.body.prioridade,
13        situacao: 'ATIVO', criadoEm: new Date().toISOString()

```

```

14   }
15   registros.push(registro)
16   return res.status(201).json(registro)
17 } catch (erro) {
18   const mensagem = erro instanceof Error ? erro.message : 'Dados inválidos.'
19   return res.status(400).json({ mensagem })
20 }
21 })
22
23 app.patch('/api/registros/:id/situacao', (req, res) => {
24   const id = Number(req.params.id)
25   const registro = registros.find(item => item.id === id)
26   if (!registro) return res.status(404).json({ mensagem: 'Não encontrado.' })
27
28   if (req.body?.situacao !== 'CANCELADO') {
29     return res.status(409).json({ mensagem: 'Transição não permitida.' })
30   }
31
32   registro.situacao = 'CANCELADO'
33   return res.json(registro)
34 })
35
36 app.listen(3000, () => console.log('http://localhost:3000'))

```

MEMÓRIA NÃO É PERSISTÊNCIA

O array desaparece quando o processo reinicia e não suporta concorrência ou auditoria reais. Em uma aplicação persistente, mantenha o mesmo contrato HTTP e troque o acesso direto ao array por serviço, repositório, consultas parametrizadas e transações.

19.7 Executar e observar

TERMINAL

```

npm install express
npm install -D typescript tsx @types/node @types/express
npx tsx src/server.ts

```

Abra <http://localhost:3000>. Ao cadastrar “Revisar documentação” com prioridade alta, a API responde com 201 Created; a interface recarrega a listagem e cria uma nova linha.

RESULTADO NO NAVEGADOR

Registros encontrados

Preparar material | Média | Ativo | Cancelar
 Revisar documentação | Alta | Ativo | Cancelar

Ao cancelar o segundo item, o navegador envia PATCH; o registro permanece na listagem com situação CANCELADO. Isso preserva o exemplo de histórico lógico sem usar exclusão física.

19.8 Leitura guiada de ponta a ponta

1. O formulário fornece strings identificadas por name.

2. `FormData` coleta e `JSON.stringify()` serializa o corpo.
3. `fetch` envia a requisição e o servidor analisa JSON com `express.json()`.
4. A rota valida tipos e regras antes de criar ou alterar o registro.
5. A API devolve status e JSON; o cliente trata sucesso e falha.
6. O estado retornado é projetado em uma tabela por criação segura de nós.
7. Busca e filtro são enviados na URL, permitindo que depois sejam executados pelo banco.
8. Em uma aplicação autenticada, middlewares ainda verificariam identidade e permissão antes da operação.

PERGUNTA: Por que a interface recarrega a lista depois de salvar?

RESPOSTA Porque o servidor é a fonte autoritativa dos dados persistidos. A resposta pode conter campos gerados e normalizados. Reconsultar simplifica a consistência; em interfaces maiores, também é possível atualizar o estado local cuidadosamente.

20 Qualidade, segurança e diagnóstico

Código correto entrega o resultado para os casos previstos e falha de maneira compreensível nos demais. Código sustentável também comunica intenção, limita efeitos colaterais e torna o fluxo observável.

20.1 Erros frequentes

Sintoma	Causa provável	Como verificar
<code>document is not defined</code>	Código de DOM executado no Node.	Separe módulo de navegador do módulo de servidor.
Resultado vira texto concatenado	Valor de campo não convertido.	Inspecione com <code>typeof</code> ; use <code>Number()</code> .
Clique não responde	Seletor devolveu <code>null</code> ou script executou cedo.	Confira seletor e carregue com <code>defer</code> .
Formulário recarrega	Listener não chamou <code>preventDefault()</code> .	Trate o evento <code>submit</code> .
<code>fetch</code> entra no sucesso em 404	Status HTTP não foi verificado.	Teste <code>response.ok</code> .
<code>req.body</code> é undefined	Parser JSON ausente ou Content-Type errado.	Use <code>express.json()</code> e cabeçalho adequado.
Import não funciona	Mistura de CommonJS/ESM ou caminho incorreto.	Confira <code>type</code> , extensão e exportação.
Array numérico "mal ordenado"	<code>sort()</code> comparou strings.	Passe <code>(a,b) => a-b</code> .
Laço nunca termina	Controle não progride em algum ramo.	Pausa no depurador e observe condição/atualização.
Resultados de busca voltam à ordem errada	Requisição antiga terminou depois da nova.	Cancele a anterior com <code>AbortController</code> ou ignore respostas obsoletas.
Cadastro aparece duplicado	Botão permitiu vários envios durante a espera.	Desabilite durante o envio e reabilite em <code>finally</code> ; proteja também no servidor.
API responde 401 ou 403	Sessão ausente ou perfil sem permissão.	Diferencie autenticação de autorização e inspecione a requisição.
Navegador bloqueia por CORS	Frontend e API usam origens diferentes sem política compatível.	Configure apenas a origem necessária no servidor.

Sintoma	Causa provável	Como verificar
TypeScript aceita o tipo, mas o dado quebra	Entrada externa foi apenas forçada com <code>as</code> .	Receba como <code>unknown</code> e valide em tempo de execução.

20.2 Segurança básica

- Use `textContent` para texto não confiável; evite `innerHTML`.
- Valide novamente no servidor; o cliente pode ser contornado.
- Não coloque segredos em JavaScript entregue ao navegador.
- Armazene senhas somente como hashes produzidos por biblioteca reconhecida.
- Verifique identidade, perfil e vínculo com o recurso em cada operação protegida.
- Não monte comandos, SQL ou caminhos diretamente com entrada do usuário.
- Use consultas parametrizadas e transações para alterações que também geram histórico.
- Não exponha mensagens internas e pilhas de erro na resposta pública.
- Mantenha dependências deliberadas e revise alertas sem atualizar cegamente.

20.3 Acessibilidade em interfaces dinâmicas

JavaScript deve preservar o comportamento nativo. Use `button` para ação; não transforme uma `div` em botão sem implementar teclado, foco e semântica. Para mensagens que mudam, `aria-live='polite'` pode anunciá-las sem mover o foco. Em modais, gerencie foco e devolva-o ao controle de origem.

20.4 Checklist antes de considerar pronto

1. Entradas vazias, inválidas, zero, negativas e limites foram testados.
2. Comparações usam `===` e conversões são explícitas.
3. Funções de domínio não dependem do DOM ou de HTTP sem necessidade.
4. Eventos usam `addEventListener`; formulários tratam `submit`.
5. Atualizações de texto não injetam HTML não confiável.
6. Requisições verificam status e apresentam estados de carregamento e erro.
7. Busca, filtros, ordenação e paginação possuem parâmetros validados.
8. O servidor valida tipos, regras, transições e permissões e retorna status coerentes.
9. Login não persiste senha; sessão e cookies usam configurações adequadas ao ambiente.
10. Cancelamentos preservam dados e histórico quando a auditoria for necessária.
11. Integrações externas têm timeout, validação da resposta e tratamento de indisponibilidade.

12. Segredos permanecem no servidor e fora do repositório.
13. TypeScript usa configuração estrita; dados externos entram como `unknown`.
14. Rotas, serviços e repositórios possuem responsabilidades separadas.
15. Console não contém erros inesperados; Network mostra requisições corretas.
16. Interface funciona com teclado, zoom e mensagens compreensíveis.

21

Revisão com respostas comentadas

As perguntas abaixo medem compreensão, não memorização. Tente responder antes de ler a linha verde.

PERGUNTA: `const` e `let` criam valores de tipos diferentes?

RESPOSTA Não. Ambos podem apontar para qualquer tipo. A diferença principal é que `let` permite reatribuição e `const` não.

PERGUNTA: Por que `5 === '5'` é falso?

RESPOSTA Porque a igualdade estrita compara valor e tipo. Um operando é `number`; o outro, `string`. Converta explicitamente se representam a mesma informação.

PERGUNTA: Qual é a saída de `console.log(1 + '2')`?

RESPOSTA `'12'`. Como um operando é `string`, `+` concatena. `1 + Number('2')` produziria `3`.

PERGUNTA: Quando usar `for...of`?

RESPOSTA Quando você quer os valores de um iterável. Para transformar arrays, métodos como `map` e `filter` podem expressar melhor a intenção.

PERGUNTA: `map` altera o array original?

RESPOSTA Não por definição: ele cria um novo array com os retornos do callback. Porém, se os itens forem objetos e o callback alterar esses objetos, as referências compartilhadas serão modificadas.

PERGUNTA: Qual a diferença entre `textContent` e `innerHTML`?

RESPOSTA `textContent` insere texto; `innerHTML` analisa marcação. Para entrada não confiável, prefira `textContent`.

PERGUNTA: Por que escutar `submit` em vez do `click` do botão?

RESPOSTA Porque o formulário pode ser enviado por teclado e outros controles. `submit` representa a ação sem depender de como ela foi iniciada.

PERGUNTA: JSON e objeto JavaScript são a mesma coisa?

RESPOSTA Não. JSON é texto com gramática limitada. `JSON.parse()` cria um valor JavaScript; `JSON.stringify()` produz texto JSON.

PERGUNTA: POST sempre significa criar um registro?

RESPOSTA Não necessariamente. É comum para criação ou processamento com corpo, mas o contrato da API define a semântica. Métodos HTTP comunicam intenção e devem ser usados de forma consistente.

PERGUNTA: FormData converte campos numéricos automaticamente?

RESPOSTA Não. Em geral, os valores coletados são strings ou arquivos. Normaliza espaços e converte números, datas, booleanos e valores opcionais explicitamente.

PERGUNTA: Onde enviar busca, filtros e ordenação em um GET?

RESPOSTA Nos parâmetros da URL. `URLSearchParams` monta a consulta com codificação adequada. O servidor ainda valida nomes, limites e valores permitidos.

PERGUNTA: Qual é a diferença entre 401 e 403?

RESPOSTA 401 indica que falta autenticação válida; 403 indica que a identidade foi reconhecida, mas não possui permissão. A resposta não deve revelar dados protegidos.

PERGUNTA: Esconder um botão impede que a operação seja executada?

RESPOSTA Não. Isso apenas orienta a interface. A API precisa verificar novamente identidade, perfil, associação ao recurso, estado atual e regra de transição.

PERGUNTA: O navegador precisa ler o cookie de uma sessão HttpOnly?

RESPOSTA Não. O navegador envia o cookie conforme sua política, mas JavaScript não acessa seu conteúdo. Isso reduz a exposição do segredo; HTTPS, SameSite, expiração e proteção contra CSRF continuam necessários.

PERGUNTA: Por que separar controller, service e repository?

RESPOSTA Para isolar HTTP, regras e persistência. A separação facilita testes, substituição do banco e leitura do fluxo. Em exemplos pequenos, as camadas podem ser compactas sem perder seus papéis.

PERGUNTA: Node.js possui a porta padrão 3000?

RESPOSTA Não. 3000 é convenção de muitos exemplos. O processo escuta a porta escolhida pelo código ou pela configuração do ambiente.

PERGUNTA: TypeScript valida automaticamente req.body?

RESPOSTA Não. Tipos desaparecem na compilação. Dados externos precisam de validação em tempo de execução antes de serem tratados como confiáveis.

PERGUNTA: Bootstrap é backend?

RESPOSTA Não. É um framework de front-end com CSS e comportamentos JavaScript para componentes. Banco de dados e regras de servidor ficam fora de seu papel.

Recurso	Função principal
<code>const / let</code>	Declara ligação sem/com reatribuição.
<code>typeof valor</code>	Informa o tipo primitivo percebido.
<code>Number() / String()</code>	Converte explicitamente.
<code>=== / !==</code>	Compara valor e tipo.
<code>?? / ?.</code>	Fallback para nulo e acesso opcional.
<code>if / switch</code>	Seleciona caminhos.
<code>for / while</code>	Repete com controle ou condição.
<code>for...of</code>	Percorre valores de iteráveis.
<code>function / =></code>	Define função tradicional ou arrow.
<code>map / filter</code>	Transforma ou seleciona itens.
<code>find / some / every</code>	Procura e testa condições.
<code>reduce</code>	Combina uma coleção em acumulador.
<code>...</code>	Expande (spread) ou reúne (rest).
<code>JSON.parse/stringify</code>	Converte entre texto JSON e valor JS.
<code>querySelector(All)</code>	Seleciona primeiro/todos os nós por CSS.
<code>textContent</code>	Lê ou define texto de um nó.
<code>classList</code>	Adiciona, remove ou alterna classes.
<code>createElement/append</code>	Cria e insere nós.
<code>DocumentFragment</code>	Agrupa nós antes da inserção no documento.
<code>replaceChildren</code>	Substitui os filhos de um elemento.
<code>addEventListener</code>	Registra resposta a evento.
<code>preventDefault</code>	Impede ação padrão do evento.
<code>FormData</code>	Coleta controles nomeados de um formulário.
<code>URLSearchParams</code>	Monta ou lê parâmetros codificados de URL.
<code>localStorage</code>	Persiste strings na origem.
<code>fetch</code>	Realiza requisição e devolve Promise.

Recurso	Função principal
<code>response.ok/status</code>	Informa se o status HTTP representa sucesso e qual código chegou.
<code>AbortController</code>	Permite cancelar operações que aceitam um sinal.
<code>async/await</code>	Organiza continuações assíncronas.
<code>export/import</code>	Expõe e consome módulos.
<code>try/catch</code>	Trata exceções síncronas e rejeições aguardadas.
<code>process.env</code>	Lê configuração do ambiente no Node.js.
<code>express.Router</code>	Agrupa rotas e middlewares relacionados.

Fontes

Materiais fornecidos

- Materiais didáticos da disciplina - referência de escopo e nível introdutório.

Material público consultado

- Barbara Calderon – estudos do Curso de JavaScript do Curso em Vídeo: <https://github.com/barbaracalderon/curso-de-javascript-do-curso-em-video>
- MDN – Guia JavaScript: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>
- MDN – DOM e eventos: https://developer.mozilla.org/pt-BR/docs/Web/API/Document_Object_Model
- MDN – Fetch API: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API
- Node.js – documentação: <https://nodejs.org/docs/latest/api/>
- Node.js – variáveis de ambiente: https://nodejs.org/api/environment_variables.html
- npm – documentação: <https://docs.npmjs.com/>
- TypeScript – handbook: <https://www.typescriptlang.org/docs/handbook/intro.html>
- TypeScript – opção strict: <https://www.typescriptlang.org/tsconfig/strict>
- Express – documentação: <https://expressjs.com/>
- Express – uso de middlewares: <https://expressjs.com/en/guide/using-middleware/>
- Express – tratamento de erros: <https://expressjs.com/en/5x/guide/error-handling/>
- Bootstrap – documentação: <https://getbootstrap.com/docs/>